Exp 5. Spam Email Detection: Build a spam email filter using text classification algorithms and perform comparative analysis. (Naïve Bayes, Logistic Regression, Support Vector Machine, Decision Tree, Random Forest)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classif
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/spam_email.csv')
df.head()
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Category  5572 non-null   object
 1   Message   5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

	Category	Message
count	5572	5572
unique	2	5157
top	ham	Sorry, I'll call later
freq	4825	30

```
df['Category']=df['Category'].map({'ham':0,'spam':1})
df.head()
```

	Category	Message
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
X=df['Message']
y=df['Category']
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
cv = TfidfVectorizer(stop_words='english')
X_train=cv.fit_transform(X_train)
X_test=cv.transform(X_test)
print("Training matrix shape:",X_train.shape)
print("Testing matrix shape:",X_test.shape)
```

```
Training matrix shape: (4457, 7526)
Testing matrix shape: (1115, 7526)
```

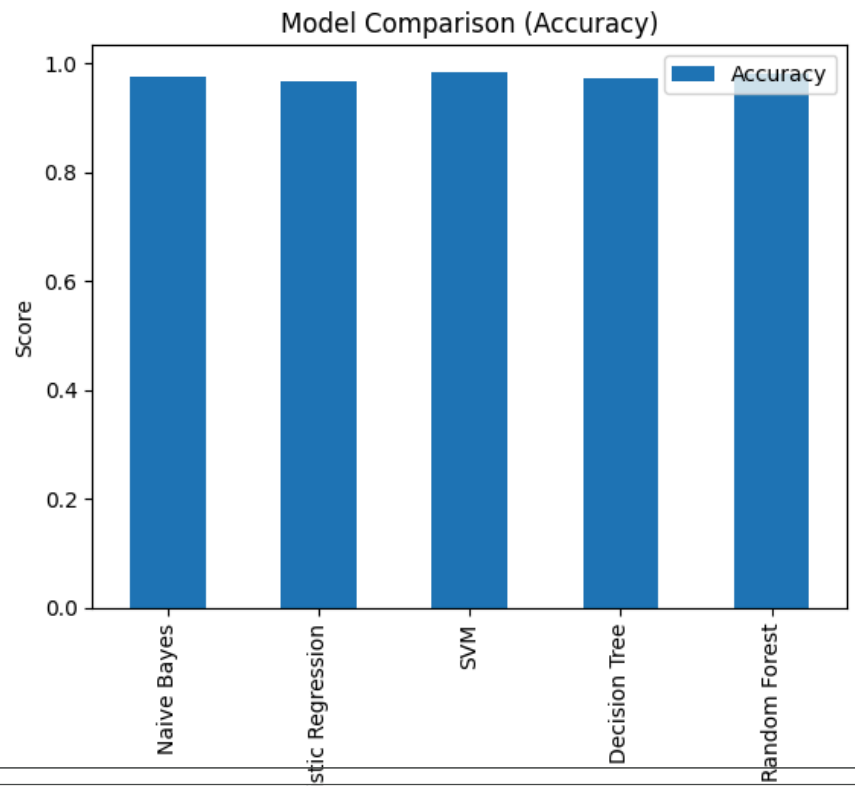
```
models={"Naive Bayes":MultinomialNB(),
        "Logistic Regression":LogisticRegression(max_iter=100),
        "SVM":SVC(),
        "Decision Tree":DecisionTreeClassifier(),
        "Random Forest":RandomForestClassifier()
        }
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred)
    rec=recall_score(y_test,pred)
    f1=f1_score(y_test,pred)
    results.append([name,acc,pre,rec,f1])
results_df=pd.DataFrame(results,columns=["Model","Accuracy","Precision","Recall","F1"])
results_df
```

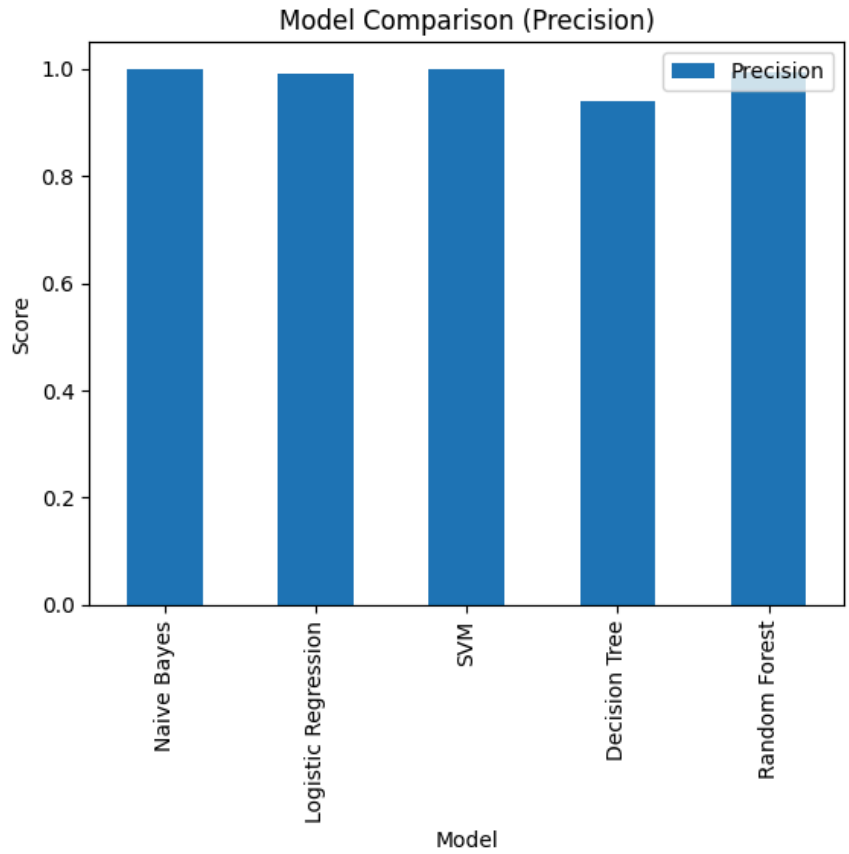
	Model	Accuracy	Precision	Recall	F1
0	Naive Bayes	0.975785	1.000000	0.83125	0.907850
1	Logistic Regression	0.965919	0.991935	0.76875	0.866197
2	SVM	0.983857	1.000000	0.88750	0.940397
3	Decision Tree	0.971300	0.938356	0.85625	0.895425
4	Random Forest	0.979372	0.992806	0.86250	0.923077

Next steps: Generate code with results_df New interactive sheet

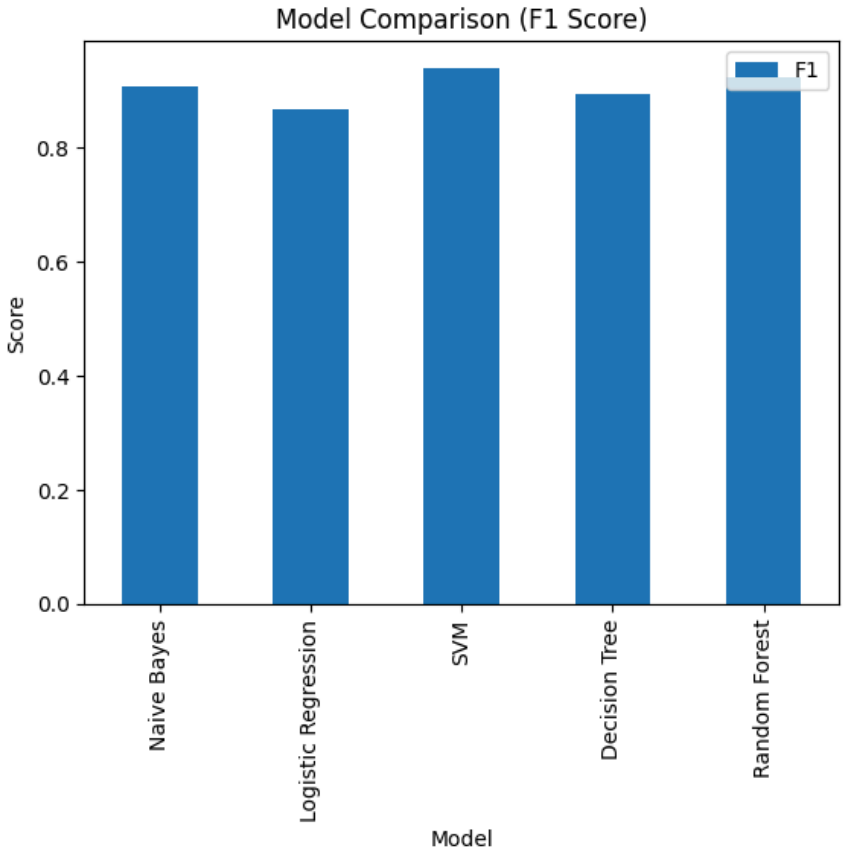
```
results_df.plot(x="Model",y="Accuracy",kind="bar")
plt.title("Model Comparison (Accuracy)")
plt.ylabel("Score")
plt.show()
```




```
results_df.plot(x="Model",y="Precision",kind="bar")
plt.title("Model Comparison (Precision)")
plt.ylabel("Score")
plt.show()
```




```
results_df.plot(x="Model",y="F1",kind="bar")
plt.title("Model Comparison (F1 Score)")
plt.ylabel("Score")
plt.show()
```



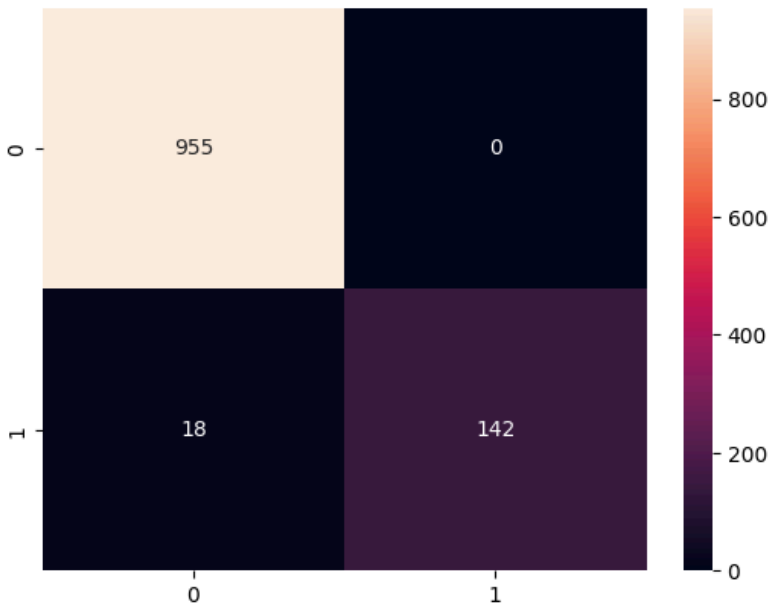

```
best_name=results_df.sort_values("F1",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: SVM
```

```
with open("spam_email_classifier.pkl","wb") as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt="d")
```

```
<Axes: >
```



```
print("ROC AUC:",roc_auc_score(y_test,probs))
```

```
ROC AUC: 0.9963612565445027
```

```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       955
           1       1.00      0.89      0.94       160

    accuracy                           0.98      1115
   macro avg       0.99      0.94      0.97      1115
weighted avg       0.98      0.98      0.98      1115
```